

Asymptotic

A source-pinned technical audit and
comparison with Wolfram Language

Independent review prepared for Vladimir Reshetnikov

9 September 2026

Correctness contracts Sparse asymptotics Inverse branches
Performance API design Reproducible engineering

Reviewed repository

<https://github.com/VladimirReshetnikov/Asymptotic>

Pinned revision

07a9781212beb2eeb9ff16aa625b50ac27974078

Package baseline: version 1.8.0; declared requirement Wolfram Language 15.0 or later.

Evidence boundary. This is a source audit with independent exact mathematical checks. A native Wolfram evaluator was attempted but unavailable. The supplied Wolfram regression tests and patch are therefore *not* reported as executed or accepted. Historical repository validation is distinguished from new evidence throughout. The accompanying Python checks were executed; their logs are included.

Abstract

The repository is substantially more than a replacement for `InverseSeries`. It combines sparse exact-weight power-log arithmetic, several inverse-expansion algorithms, exact inverse cores, finite exponential sectors, oscillatory coefficients, native special-function adapters, explicit remainder transport, local branch evidence, and a separate exact interval-certificate subsystem. Its strongest contribution is a persistent, inspectable expansion object carrying information that an ordinary displayed approximation loses.

The most actionable findings are a real-branch guard bypass for nested fractional-power observables, an eager native-series conversion that can turn two sparse rational-power blocks into more than one hundred million dense coefficient slots, and a provably redundant coefficient-composition loop. Additional concerns involve expression-provenance growth, trusted raw association payloads, fragmented resource budgets, API conventions, and the distinction between focused validation and full release acceptance. Several possible proof gaps are identified as review targets rather than asserted public failures.

The recommended strategy is to strengthen common invariants and deployment evidence before adding more special-function families. Wolfram Language should remain a computational backend and comparison baseline, not be portrayed as limited to Taylor series. A small, source-hash-guarded hardening patch generator, native regression specifications, diagnostic benchmarks, and independently executed mathematical checks accompany the report.

Contents

1	Executive assessment	4
1.1	Overall judgment	4
1.2	Decision summary	4
1.3	What is not being claimed	5
2	Snapshot, sources, and reproducibility	5
2.1	Repository organization	5
2.2	Evidence categories	6
2.3	Executed independent checks	6
3	Architecture and mathematical contracts	6
3.1	The ordinary sparse power-log algebra	6
3.2	Inverse construction and the exact-weight frontier	7
3.3	Why input remainder transport needs more than algebra	7
3.4	Different scales are not interchangeable	7
4	Comparison with built-in Wolfram Language	8
4.1	The correct baseline	8
4.2	Reversion and inverse branches	9
4.3	Arithmetic and differentiation	9
4.4	What an honest numerical comparison would measure	10
5	Correctness finding: nested real-power guard bypass	10

5.1	A01: A pure remainder acquires an unjustified real fractional power	10
5.1.1	The invariant	10
5.1.2	A minimal public regression specification	11
5.1.3	Control-flow proof	11
5.1.4	Proposed repair	11
6	Performance finding: sparse-to-dense native conversion	12
6.1	A02: Two sparse rational powers can allocate a hundred-million-slot array	12
6.1.1	Mechanism	12
6.1.2	Exact witness and scale of the problem	12
6.1.3	Recommended fix and its limits	13
6.1.4	Native format is an adapter, not the internal algebra	13
7	Performance finding: a zero coefficient tail is not terminated	13
7.1	A03: The composition recurrence keeps multiplying after permanent termination	13
8	Structural, resource, and validation findings	14
8.1	A04: Replayable recipes can expand as trees rather than shared graphs	14
8.2	A05: The raw association constructor is a trusted boundary	15
8.3	A06: Resource control is local and heterogeneous	15
8.4	A07: Focused validation is not full release acceptance	15
8.5	A08: A common head hides several different request contracts	16
9	Additional proof obligations and unconfirmed probes	17
9.1	R01: real finite phases do not imply real uncertain phases	17
9.2	R02: real scalar validation differs between entry paths	17
9.3	R03: unknown native tails need an honest logarithmic degree	17
9.4	Why unconfirmed findings are kept separate	18
10	Mathematical safeguards worth preserving	18
10.1	Differentiating a Big-O term requires a contract	18
10.2	Exponentiating a relative approximation can be wrong	18
10.3	Oscillatory coefficients are not invertible units by default	19
10.4	A zero truncated residual is not automatically exact termination	19
10.5	Exact interval certification: a separate, narrower achievement	19
11	Performance analysis and a fair benchmark program	20
11.1	Separate representation complexity from coefficient complexity	20
11.2	What the repository's recorded benchmarks actually support	20

11.3 Recommended workload matrix	20
12 API and representation redesign proposal	21
12.1 A shared result contract	21
12.2 Centralize invariant enforcement	21
12.3 Improve diagnostics without weakening conservatism	22
12.4 Numerical evaluation should expose what it discards	22
13 Missing functionality and development opportunities	22
13.1 A safe generalized-series integration operation	22
13.2 Uniform parameter regimes and transition points	22
13.3 Multivariate and coupled implicit systems	23
13.4 Broader coefficient algebras and transseries	23
13.5 Extending numerical certification	23
13.6 Distribution and compatibility	23
14 Test strategy and prioritized roadmap	24
14.1 Test mathematical invariants, not only example strings	24
14.2 Roadmap with acceptance criteria	24
15 Conclusions	25
A Review coverage and source map	25
B Independent inverse-coefficient oracle	27
C Bundle contents and execution status	27
C.1 Applying the proposal in a disposable checkout	28
C.2 Native runner safeguards	28
C.3 Building the article	28

1 Executive assessment

1.1 Overall judgment

The package has a coherent and useful specialization: *real, source- and target-aware asymptotic calculations whose truncation, coefficient scale, and remainder can survive subsequent operations*. It exposes ordinary arithmetic on `GeneralizedSeries`, but also explicit operations for composition, refinement, differentiation, inverse residuals, and certification. There is meaningful engineering around branch selection, exact coefficients, generated standalone loading, and historical validation. Treating this repository as a small experimental reversion script would miss most of its current design.[1, 3, 6, 5, 9]

Conversely, breadth is not the same as a single closed transseries algebra. Different families use different coefficient rings and truncation meanings, and some mixed-family operations deliberately fall back to composite absolute-error envelopes. Some native special functions are candidates for a guarded adapter rather than universally supported inputs. The result is best understood as a collection of cooperating, partially overlapping algebras with a common wrapper.[3, 7, 11]

Priority recommendation: fix the common low-level branch invariant and dense-export allocation first; then establish a reproducible full-suite release gate and a stable typed contract for scale, domain, error, and provenance. New families should be admitted through that contract.

1.2 Decision summary

ID	Severity / evidence	Finding	Recommended action
A01	High; source-proven guard gap	Nested observables reach a fractional-power path that accepts a sign-unknown pure remainder. Native public output is predicted, not measured.	Centralize the guard in <code>fwdPower</code> ; test nesting.
A02	High; exact allocation calculation	Eager conversion to <code>SeriesData</code> allocates a dense rational-exponent lattice without a span budget.	Guard now; make native export lazy later.
A03	Low-medium; exact recurrence proof	A zero composition coefficient causes <code>Continue</code> , although every later coefficient is zero.	Replace with <code>Break</code> ; compare output and operation counts.
A04	Medium; structural risk	Whole operands in binary refinement recipes can produce exponential serialized provenance. Physical memory growth is unmeasured.	Use a shared recipe graph and bounded export.
A05	Medium; trust-boundary risk	Raw result associations can contain mutually inconsistent displayed expressions and internal jets.	Version and validate external object construction.
A06	Medium; observed design limitation	Budgets have several meanings; some expansion/simplification work is not covered by a single global resource policy.	Separate work, storage, proof-time, and output budgets.
A07	Medium; documented release gap	Strong focused results are not a full semantic release-suite run; the inspected CI workflow checks standalone freshness.	Add native semantic release acceptance.

ID	Severity / evidence	evi- Finding	Recommended action
A08	Medium; UX compatibility	Order, power observables, assumptions, numerical evaluation, and migration have nonuniform or surprising meanings.	Structured requests, better summaries, explicit compatibility policy.
R01– R03	Unconfirmed re-view targets	Complex intermediate errors in trigonometric native trees; real scalar consistency; unresolved native-tail logarithmic degree.	Targeted native probes and invariant proofs, not speculative hotfixes.

1.3 What is not being claimed

No native kernel timings were measured for this audit. No count of passing upstream tests is presented as a new test run. The report does not claim that all 38 kernel source files, every mathematical proof in the repository’s article, every notebook, or every historical artifact received a line-by-line review. The reviewed central execution paths, documentation, representative tests, and source ranges are identified in Appendix A. Broad feature descriptions outside those paths are explicitly grounded in the current guide. No repository changes or external issue submissions were made.

The absence of a universal algorithm for every real or complex inverse is not itself a bug. Nor is a conservative **Failure** an incorrect result when a branch, order, or error condition cannot be established. The relevant questions are whether a successful result meets its stated contract and whether a rejection accurately explains the missing condition.

2 Snapshot, sources, and reproducibility

2.1 Repository organization

The pinned tree contains a canonical modular package under `AsymptoticInverse/Kernel`, a generated repository-root `AsymptoticInverse.wl`, a paclet layout, examples, tests, a mathematical article, a user guide, validation records, and historical development material. The kernel inventory contains 38 `.wl` files plus `init.m`. The standalone distribution is 573,940 bytes in the reviewed tree. These are inventory facts, not a count of independently audited algorithms.[2, 15, 1]

The modular implementation is the source of truth. The generated file carries module provenance and is checked by a deterministic builder. Rebuilding and checking the standalone distribution after a source change is part of the intended development workflow. The development notes explicitly distinguish current reader-facing documentation from preserved historical plans and reports. Accordingly, this audit does not label a feature missing merely because an old plan lists it as future work.[15, 16]

The reviewed core file has Git blob identifier

ea9eaf4a11130e922ac4fb3faae37fd8e8d29643.

The supplied patch generator verifies this content identity, after normalizing a CRLF worktree to LF, before producing a proposed change. A commit pin identifies the whole snapshot; the blob identifier identifies the particular edited file. Neither should be confused with a SHA-256 digest of a downloaded release archive.

2.2 Evidence categories

Four categories are used. A *source-proven* result follows from explicit code paths and a stated mathematical invariant. A *mathematical witness* is checked independently of the implementation. A *historical validation claim* is a repository record with its own scope and version. A *native probe* is a supplied test that remains unexecuted here.

The Wolfram connector failed even on a basic version-and-arithmetic query. Direct checkout/download access in the container was also unavailable; repository material was read through the GitHub connector. Therefore, the complete original package was not installed in the container, the upstream builder was not rerun here, and the patch was not applied to a complete local checkout. The patch-editing machinery was tested against small exact source-anchor fixtures; that is much weaker than a native integration test and is labelled accordingly.

Current official Wolfram reference pages were consulted on 9 September 2026. Documentation-based capabilities and actual behavior of a particular installed kernel are different kinds of evidence. The benchmark harness records the native kernel version when the reader runs it; this report does not infer the current latest Wolfram release from the repository’s minimum requirement.

2.3 Executed independent checks

The bundle’s `independent_checks.py` uses exact rational arithmetic and SymPy 1.14.0. It verifies the rational-lattice span in A02, the complex square-root witness in A01, five generalized inverse coefficients by independent composition, a zero-tail recurrence, an equal-weight collision, and a derivative-bound counterexample. Eight Python artifact tests check the planner and fail-closed patch machinery. The final logs report success. These checks establish the stated mathematical or tooling properties; they do not establish that the Wolfram package evaluates as predicted.

3 Architecture and mathematical contracts

3.1 The ordinary sparse power–log algebra

Let $u \rightarrow 0^+$ be the positive local coordinate. An ordinary expansion is a finite sum

$$J(u) = \sum_{\beta \in B} u^\beta P_\beta(\log u) + \mathcal{O}\left(u^\rho(1 + |\log u|)^k\right), \quad (1)$$

where B is a finite ordered set of exact real exponents and each P_β is a polynomial in the logarithm. The core uses precision-tracked triples consisting of rows, a remainder power, and a logarithmic degree. Rows at equal weights are merged before truncation.[4, 5]

For finite source points the positive coordinate is either $x - x_0$ or $x_0 - x$; at signed infinities it is $1/x$ or $-1/x$. This is not cosmetic: a real noninteger power, the sign of an eventual leading term, and the correct logarithm all depend on the selected positive coordinate. The guide specifies these conventions and an *exclusive* cutoff. A block at exactly the cutoff belongs to the omitted part.[3, 4]

The object layer may represent

$$b(y) + C(y)(J(w(y)) + \mathcal{O}(R(w(y)))), \quad (2)$$

with an exact offset b , exact carrier C , and a small coordinate w . Its absolute error is then controlled by $|C(y)|R(w(y))$, not merely $R(w(y))$. The series-calculus code explicitly preserves this distinction.[5]

3.2 Inverse construction and the exact-weight frontier

After normalization, the ordinary inverse engine works with a model of the form

$$y - y_0 = au^p \left(1 + \sum_{j=1}^m u^{\delta_j} P_j(\log u) \right), \quad a \neq 0, \quad p \neq 0, \quad \delta_j > 0. \quad (3)$$

The source and target signs are selected so the uniformizer is real. Corrections live in the additive semigroup generated by the positive gaps δ_j . The repository offers direct Lagrange coefficients, grouped Lagrange computation, and Newton refinement with precision doubling. The incremental state also carries a bounded cache of powers of logarithmic polynomials; this cache already exists and should not be proposed as though absent.[4, 8, 3]

For a multi-index $\mathbf{k}$, write $n = |\mathbf{k}|$ and $\omega = \mathbf{k} \cdot \boldsymbol{\delta}$. For the observable u^r , the inspected coefficient routine applies

$$\frac{(-1)^{n_r}}{p^n \prod_j k_j!} \prod_{j=1}^{n-1} (\partial_L + r + \omega + pj) \prod_i P_i(L)^{k_i}, \quad n \geq 1. \quad (4)$$

The zero-index coefficient is one. This formula explains both why equal weights must be merged and why polynomial-degree growth matters independently of the number of weights. The grouped route combines contributions before applying the common differential operator.[8]

The incremental frontier is correctly organized to consume an entire equal-weight layer, not just the first index at a weight. With gaps 1 and 2, weight 2 has indices (2, 0) and (0, 1). Their contributions can cancel, so an implementation counting the first one as a completed nonzero block would be wrong. The reviewed implementation explicitly avoids that mistake. Newton refinement likewise checks that the normalized symbolic residual vanishes below its claimed precision.[8]

3.3 Why input remainder transport needs more than algebra

Suppose the model is perturbed by $\mathcal{O}(u^\rho L^k)$, where here $L = 1 + |\log u|$, and $f'(u) \asymp u^{p-1}$. A mean-value or local inverse argument gives an inverse displacement of order

$$\Delta u = \mathcal{O}(u^{\rho-p+1} L^k). \quad (5)$$

For the observable u^r this becomes $\mathcal{O}(u^{\rho-p+r} L^k)$. Translating to the target's positive small coordinate yields the exponent cap used by the ordinary engine. This derivation presupposes the derivative control needed to preserve the local branch, not just a value bound. The guide states that a declared `"InputRemainder" -> {rho,k}` includes matching first-derivative order. That is a significant contract, and must be visible to callers.[4, 3]

The declaration is not a numerical constant or a universal certificate over every function with the same displayed prefix. It is an assumption about the admitted remainder class. The separate certificate subsystem explicitly declines to certify an unspecified input-remainder family.[9]

3.4 Different scales are not interchangeable

The guide describes several additional families. Lambert inverses use reciprocal-logarithmic corrections around a retained prefactor. Increasing Gamma/LogGamma inverses use an exact Lambert core with complete reciprocal-log coefficient polynomials at each inverse-core power. Barnes inverses use a related shifted-log coefficient coordinate. Source and target charts transport these expansions through exact

transformations. Flat inverses use finite exponential-sector truncations; Fourier inverses use oscillatory logarithmic coefficients with separate frequency budgets.[3]

The special-function adapter layer is particularly important to interpret correctly. The broad native forward adapter requests a structured Wolfram `Series` expansion with `Analytic -> False`, transports its error through a supported expression tree, proves an appropriate real source domain, and projects to a real expression only under that evidence. A function being a candidate for this route does not imply support for every parameter regime or endpoint.[11]

Incompatible coefficient rings are sometimes combined using an envelope rather than a falsely ordered single series. For example, for independent errors,

$$(A + \mathcal{O}(R)) + (B + \mathcal{O}(S)) = A + B + \mathcal{O}(R + S), \quad (6)$$

$$(A + \mathcal{O}(R))(B + \mathcal{O}(S)) = AB + \mathcal{O}(|A|S + |B|R + RS). \quad (7)$$

The composite representation deliberately retains separate error scales and refuses a single exponent cutoff when one has not been justified. This is a sound fallback, not evidence of a complete mixed transseries field.[7, 6]

4 Comparison with built-in Wolfram Language

4.1 The correct baseline

The native baseline includes `Asymptotic` and `AsymptoticSolve`, alongside conventional series tools. Wolfram’s documented asymptotic functions infer scales that can extend beyond Taylor, Laurent, and Puiseux forms; `AsymptoticSolve` supports implicit equations and systems, real-only solutions, and finite or infinite centers. Accordingly, the assertion that built-in Wolfram Language “only handles rational power series” would be false.[22, 23]

There is a narrower, useful representation distinction: one `SeriesData` object uses a common rational power lattice. Wolfram documentation also explicitly shows logarithmic coefficients, symbolic powers outside `SeriesData`, and rapidly growing or oscillatory factors outside it. The repository’s sparse real-weight object and error metadata are a different representation choice, not proof that Wolfram cannot compute an expression involving those scales.[19]

Task	Built-in baseline	Repository contribution / limitation
Local power series	<code>Series</code> , <code>SeriesData</code> , coefficient operations; native arithmetic, differentiation, integration.	Sparse exact real weights, explicit logarithmic error degrees and retained coordinate metadata. Not a replacement for every native series task.
Reversion	<code>InverseSeries</code> reverses appropriately structured series, including ramified examples in the documentation.	Direct generalized power–log inverse coefficients, several algorithms, input-order caps, source-side selection, refinement state.
Implicit asymptotics	<code>AsymptoticSolve</code> handles equations and systems with automatically inferred scales.	More specialized univariate inverse workflow, structured result and explicit model provenance; not general system solving.

Task	Built-in baseline	Repository contribution / limitation
Composition	ComposeSeries and series substitution require compatible limiting points.	Positive-coordinate checks and transported errors across admitted generalized scales; not every pair of package families is composable.
Special functions	Series and Asymptotic already have substantial built-in special-function knowledge.	Guarded import, carrier decomposition, selected identities/domains, and retained remainders; much of the broad forward adapter delegates to native Series .
Transcendental cores	Built-in ProductLog , inverse functions, simplification, and asymptotics are available building blocks.	Preserves exact inverse cores rather than flattening all slow factors into a finite elementary approximation.
Error checks	AsymptoticLess tests an asymptotic relation; ordinary numerical root finding is another separate facility.	Formal residuals, numerical checks, and exact rational interval certificates are separate APIs with different scopes.
Calculus beyond series	AsymptoticIntegrate directly addresses asymptotics of integrals.	The current public operation index has no general SeriesIntegrate ; integration is a development opportunity, not a reason to replace the native integral engine.

The built-in columns follow the official references; package behavior follows the guide and the inspected implementation paths. The table is a capability map, not an empirical winner/loser benchmark.[18, 19, 20, 21, 23, 22, 24, 25, 3, 5, 11]

4.2 Reversion and inverse branches

InverseSeries is the natural first comparison for ordinary local reversion; its documentation includes examples such as reversing $x \sin x$ and explicitly supplied ramified series. One should not infer that a nonzero ordinary linear coefficient is universally necessary for this built-in function.[20]

For an implicit inverse, **AsymptoticSolve** is the stronger baseline. In the package, the source endpoint and side explicitly select a local branch, and inference for unevaluated **InverseFunction** nodes records whether candidate enumeration is complete. A bounded candidate search is not treated as a completeness proof. This inspectable evidence is valuable even when both approaches ultimately produce the same finite coefficients.[23, 10]

A recommended comparison case is the inverse of $x + x^{\sqrt{2}}$ near 0^+ . The package can state a nonzero-block request in its chosen real-weight scale. The native harness asks **AsymptoticSolve** for a corresponding term goal, but the two requests are not automatically equivalent until the actual returned scales and truncations are inspected. No current-kernel success or failure for this example is asserted here.

4.3 Arithmetic and differentiation

Native **SeriesData** already supports rich arithmetic and calculus. The repository’s important distinction is its explicitly declared remainder class and derivative-order contract. For a general Poincaré remainder,

differentiation is not justified merely because an expression is printable with a Big-O term. The package’s refusal in that case is desirable and should be preserved.[19, 5]

Composition must align the outer endpoint with the inner limit, in both systems. The repository additionally deals with different positive coordinates, exact carriers, and branches of real powers. These are precisely the paths on which guard duplication becomes dangerous: agreement between a direct operation and the same operation nested in an observable should be a tested invariant.[21, 5]

4.4 What an honest numerical comparison would measure

Measure coefficient generation, representation conversion, subsequent arithmetic, refinement, and numeric evaluation separately. Native and package outputs may encode different information. Comparing a bare approximation against a structured object containing a model, branch evidence, a coefficient cache, and replayable provenance is not an equal output-work comparison. Conversely, counting that extra metadata as an advantage is inappropriate when the caller only needs a finite numerical approximation.

For ordinary integer-order Taylor tasks, package cutoff h corresponds to retaining powers strictly below h ; a native **Series** order of $h - 1$ is the natural comparison when no cancellations or family-specific conventions intervene. For special-function carriers, logarithmic blocks, inverse cores, and sector depth, explicitly normalize the error target instead of equating the integer arguments. **Asymptotic**’s order is not generally synonymous with polynomial degree.[3, 18, 22]

The supplied benchmark records version, platform, entry-file hash, bounded timings, result size, failures, and unresolved native calls. It intentionally records **SemanticEquivalenceVerified** $\rightarrow$ **False**. A failed or unevaluated native result is not a very fast successful computation. Likewise, a package failure is not evidence that the mathematical problem has no expansion.

5 Correctness finding: nested real-power guard bypass

5.1 A01: A pure remainder acquires an unjustified real fractional power

Classification: high severity; source-level guard gap, mathematical witness verified independently, public evaluation not run.

5.1.1 The invariant

A result $s(u) = \mathcal{O}(u^2)$ alone does not establish $s(u) \geq 0$. Hence it cannot justify a real-valued square root. The direct **seriesPower** path recognizes this: a noninteger power of a nonexact jet with no known leading row raises **UnknownLeadingTerm**. The lower-level **fwdPower** path does not consistently enforce the same condition.[5, **seriesPower**][4, lines 369–387]

The discrepancy matters because **seriesJetApply**, used for general observables, recursively dispatches a **Power** node directly to **fwdPower**. The top-level shortcut to the guarded public power path applies only when the entire observable has the special form z^r . Adding an outer sum, or nesting inside another regular function, takes a different route.[5, **seriesJetApply**, **SeriesObservable**]

5.1.2 A minimal public regression specification

```
Clear[x, z];
pure = AsymptoticExpansion[-x^2, {x, 0, 1}];
SeriesPower[pure, 1/2]
SeriesObservable[pure, 1 + Sqrt[z], z]
```

The first construction retains no finite term and records an $\mathcal{O}(x^2)$ remainder on $x \rightarrow 0^+$. Direct power application rejects the missing real branch. Source tracing predicts that the nested observable instead reaches the positive-exponent empty-jet case and returns a generalized result with finite part 1 and an order- x remainder.

The underlying explicit function provides the witness:

$$1 + \sqrt{-x^2} = 1 + ix, \quad x > 0, \quad (8)$$

using the principal complex square root. It is not a real-valued observable. An arbitrary member of the magnitude-only remainder class could also have either sign, so the error metadata by itself cannot repair the missing branch condition.

Precise scope of the defect. The complex magnitude statement $1 + ix = 1 + \mathcal{O}(x)$ is true. The defect is the *real-branch guarantee and consistency of the API*, not a claim that this particular complex Big-O estimate is false. This distinction is important when evaluating the regression.

5.1.3 Control-flow proof

In the inspected `fwdPower`, nonnegative integer powers are handled first. If the remaining input has no finite rows and the exponent is positive, the routine immediately returns a pure remainder with multiplied exponent and a ceiling-adjusted logarithmic degree. It does not require a positive leading coefficient because there is no leading coefficient to inspect. The later positivity check is reached only for nonempty jets.[4, lines 370–386]

For the nested observable, the recursive sum applies that rule to the square root of the empty jet and adds the exact constant one. No later real-branch proof is performed by `seriesMake`. This establishes the guard bypass from the source. A native run is still necessary to confirm the complete public evaluation and to detect any unforeseen evaluator interaction.

5.1.4 Proposed repair

Enforce the invariant at the lowest shared precision-algebra operation:

```
If[T === {},
  If[! IntegerQ[rr] && P != Infinity,
    fail["UnknownLeadingTerm",
      "A pure remainder does not establish the real branch required by a noninteger power
      ."]];
  (* existing empty-jet handling follows *)
]
```

Do not require a known leading term for a positive integer power: it is defined for either real sign. Do not reject the exact zero function raised to a positive noninteger exponent. Exact zero is different from an unknown remainder, which is why the precision test is needed. Negative powers must continue to reject a sign-unknown or possibly zero remainder.

The tests include direct, sum-nested, and deeper-nested square roots, a positive known leading monomial, and exact zero. Moving the failure into `fwdPower` may trigger the existing forward working-order retry mechanism in cases where additional terms could resolve the sign. That is preferable to silently guessing, but its performance and termination must be checked against the full suite.

6 Performance finding: sparse-to-dense native conversion

6.1 A02: Two sparse rational powers can allocate a hundred-million-slot array

Classification: high severity; source allocation path and exact size calculation established.

6.1.1 Mechanism

Both `makeSeriesData` and `makeInverseSeriesData` compute a common denominator from the retained exponents and the remainder frontier, then allocate

```
den = LCM @@ (Denominator /@ Append[exps, remData[[1]]]);
nmin = (* least retained exponent *) den;
nmax = remData[[1]] den;
coeffs = Table[0, {nmax - nmin}];
```

The converter receives no `MaxTerms` argument and performs no dense-span check. It is called while constructing the expansion object, rather than only when a user explicitly requests the native representation. Thus an optional interoperability field can dominate, or prevent, an otherwise cheap sparse computation.[\[4, lines 647–660 and `makeInverseSeriesData`\]](#)

6.1.2 Exact witness and scale of the problem

Consider the supported finite expression

$$f(x) = x^{1/10007} + x^{1/10009} + x, \quad x \rightarrow 0^+, \quad (9)$$

with exclusive cutoff 1. Only two rows are retained; the omitted x supplies remainder frontier 1. Because 10007 and 10009 are coprime,

$$d = \text{lcm}(10007, 10009) = 100160063, \quad (10)$$

$$n_{\min} = 10007, \quad n_{\max} = 100160063, \quad (11)$$

$$N = n_{\max} - n_{\min} = \boxed{100150056}. \quad (12)$$

The planner included in the bundle verifies these integers without allocating the array.

An illustrative eight-byte-per-slot storage model gives 801,200,448 bytes before additional structure, input, or symbolic overhead. This is *not a measured Wolfram memory footprint or a guaranteed lower bound for every implementation*: zero sharing, packing, and internal representations can change storage. The decisive source fact is the demand to construct a list of 100,150,056 entries. Increasing the two coprime denominators worsens it rapidly.

The public triggering expression is

```
AsymptoticExpansion[
  x^(1/10007) + x^(1/10009) + x, {x, 0, 1},
  "MaxTerms" -> 100]
```

It should not be run on the unpatched implementation merely to confirm the allocation. The supplied large-denominator native tests are in a separate, patched-only suite with a second memory guard. A smaller 101,103 example has 10,302 slots and is suitable for a bounded diagnostic benchmark.

6.1.3 Recommended fix and its limits

The minimal patch checks the projected span before `Table` and returns

```
Missing["DenseRepresentationTooLarge",
  <|"RequiredSlots" -> nmax - nmin, "SlotLimit" -> 20000|>]
```

for the `"SeriesData"` property while preserving the useful sparse result. The same repair is required in both forward and inverse converters. Failing the entire expansion would be inferior when the sparse computation is already valid.

The supplied fixed cap is a conservative stopgap, *not* a complete implementation of the user-specified `MaxTerms` contract. The durable API should make native conversion lazy and expose a separate `"MaxNativeSeriesSlots"` or byte/work budget. Before conversion it should inspect the lattice span, direction, logarithmic error semantics, and representation compatibility. Exact results, unsupported irrational weights, and expensive rational lattices should have distinct reasons for unavailable conversion.

6.1.4 Native format is an adapter, not the internal algebra

A common rational lattice is appropriate for `SeriesData`; that is its documented representation. The problem is not that Wolfram has a rational-lattice format. It is the repository's eager decision to densify sparse data into that format. A sparse exact-weight package should remain sparse when native interoperability is unavailable or expensive.^[19]

7 Performance finding: a zero coefficient tail is not terminated

7.1 A03: The composition recurrence keeps multiplying after permanent termination

Classification: low–medium severity; exact optimization, not a wrong coefficient.

The routine `jetComposeBlock` expands

$$(1 + U)^a P(L + \log(1 + U)) = \sum_{k \geq 0} Q_k(L) U^k, \quad (13)$$

using

$$Q_0 = P, \quad Q_{k+1} = \frac{(a - k)Q_k + Q'_k}{k + 1}. \quad (14)$$

After updating Q_k and multiplying the current power of U , the source uses `Continue[]` when the new coefficient is provably zero.^[4, lines 251–263]

Proposition 7.1. *If Q_k is identically zero in the coefficient polynomial algebra, then Q_j is identically zero for every $j > k$.*

Proof. Both multiplication by $a - k$ and differentiation send the zero polynomial to zero. Equation (14) therefore sends $Q_k = 0$ to $Q_{k+1} = 0$, and induction proves the assertion. $\square$

For $a = 0$ and $P = 1$, the answer is exactly one, but the loop can continue generating powers $U, U^2, \dots$ until the cutoff is reached. Replacing this specific `Continue[]` with `Break[]` avoids all subsequent work and preserves the finite result. The proposed patch retains the one multiplication already performed before the zero test; a further local rearrangement could eliminate that multiplication too, but is not needed for the minimal repair.

The optimization depends on the zero predicate proving equality in the polynomial coefficient algebra. It must not be generalized to “a numerical sample was zero” or to coefficients known only to vanish at one value of L . The included exact recurrence check and native result-control specification test the intended algebraic situation. No speedup factor is claimed without native timing.

8 Structural, resource, and validation findings

8.1 A04: Replayable recipes can expand as trees rather than shared graphs

Classification: medium severity; structural serialization risk, runtime memory unmeasured.

`seriesMake` retains a `SeriesRecipe`, and binary operations include their complete operand objects in that recipe. Retaining enough information to refine a derived result is useful; simply deleting provenance would remove a documented capability. The problem is the representation of shared dependencies.[5, `seriesMake`, `seriesBinary`]

Consider repeatedly applying an explicit binary operation to the same value:

```
s = AsymptoticExpansion[1, {x, 0, 2}];
Do[s = SeriesMultiply[s, s], {n}]
```

The mathematical result stays one. If each recipe embeds both prior operands, the logical recipe tree has the recurrence $T_{n+1} = 2T_n + 1$ and hence $T_n = 2^{n+1} - 1$ nodes for a unit-sized initial node. At depth 20 that abstract tree has 2,097,151 nodes. This is an exact structural model, not a measured `ByteCount` for the package.

Wolfram’s runtime may share subexpressions internally, so claiming exponential physical memory from this observation alone would overstate the evidence. Serialization, expression traversal, hashing, or a naïve replay can nevertheless revisit the same logical dependency many times. The bounded inspection probe records `LeafCount`, `ByteCount`, and `InputForm` length separately, precisely because they measure different things.

A better representation is an immutable directed acyclic graph: operation nodes refer to content-identified operands, and a result carries a root identifier plus a compact provenance store. Cache keys must include the coefficient algebra, assumptions, approach, truncation semantics, and implementation/schema version. Refinement should memoize a node by both its identifier and requested precision. Existing state-local polynomial caches are compatible with this design; they do not solve recipe sharing by themselves.[8, 5]

8.2 A05: The raw association constructor is a trusted boundary

Classification: medium severity; extension and serialization risk, not an ordinary-input exploit.

The public head `GeneralizedSeries` wraps an association. `Normal` reads its `"Expression"` field; series calculus can instead trust a stored `"SeriesRepresentation"` without checking consistency. A caller who manually changes the former but retains the latter can make displayed and arithmetic behavior disagree.[4, 5]

This is not presented as a bug in the package's ordinary constructors, which normally build their own data. It is a risk for extension authors, saved expressions, migrations, and user code that manipulates result associations. The inspection probe deliberately constructs malformed data to expose the boundary; such a probe is not a claim that malformed objects are supported inputs.

Define a versioned external constructor or importer that checks required fields, variable scope, positive-coordinate metadata, coefficient-ring membership, exponent ordering, remainder consistency, and the relationship between finite expression and internal representation. Keep a separate trusted internal constructor for performance. A successful validation may cache an invariant token, but that token must not survive arbitrary payload mutation. Existing read-only interpretation boxes are useful against accidental notebook editing; they are not a replacement for data validation.[4]

8.3 A06: Resource control is local and heterogeneous

Classification: medium severity; observed API limitation and performance risk.

The single spelling `"MaxTerms"` controls different quantities in different paths: leaves, retained rows, multi-indices plus boundary entries, Taylor order, and intermediate products. The incremental cache intentionally shares spare index budget; this is a thoughtful local policy. Other paths have fixed Taylor-order caps or independent proof timeouts. The native adapter also performs symbolic expansion and simplification around its bounded native series call.[4, 8, 11, 3]

This is not inherently wrong, but the option is not a universal memory or wall-clock guarantee. A small number of logarithmic-polynomial coefficients can have enormous degree or integer height. A factored expression can expand into exponentially many terms before an output-size check. Repeated short proof calls can consume a large aggregate budget, while a helper without a local timeout can dominate a request.

Use distinct request budgets for retained blocks, generated candidates, coefficient degree, coefficient bit-size, native dense slots, expression leaves, total proof time, and aggregate work. Report the exhausted quantity and the phase that exhausted it. Add projected-size checks before expansion, not only after an expanded result exists. A resource failure should preserve the best sound partial object only when its remainder contract still holds; an unfinished coefficient must not be promoted to a complete block.

For inverse frontier discovery, the inspected implementation scans the boundary to collect a layer and sorts its weights to obtain the next one. A priority structure with exact weight comparisons can reduce repeated ordering work, but it must consume *all* collisions at a weight and preserve the index-budget invariant. No claim of an asymptotically optimal or measured faster implementation is made here.[8]

8.4 A07: Focused validation is not full release acceptance

Classification: medium severity; explicitly documented evidence gap.

The current validation summary is unusually careful: it records focused test sets, fresh-kernel conditions, package-source identities, standalone checks, and benchmark limitations. Its current section reports

305 passing tests in 18 selected files, along with builder and standalone checks, and explicitly says that the full package suite was not run. Earlier sections describe other focused subsets, including 316 tests during refactoring. These counts overlap and must not be summed into a fictitious total.[14]

The inspected GitHub Actions workflow runs the standalone builder in check mode and Python builder tests. It does not run a Wolfram semantic suite. This is a statement about that workflow, not an unsupported claim that no other validation process exists anywhere. A green freshness check establishes distribution consistency, not correctness of branch selection or remainder transport.[16]

The arithmetic test excerpt already checks independent low-order coefficients, exactness restrictions, real fractional-power branches, cancellation of displayed terms without loss of an unknown tail, coordinate compatibility, and refinement replay. Those are meaningful tests, not superficial smoke tests. The A01 finding motivates extending their *cross-entry-path* coverage: a direct guard and its nested-observable counterpart should be tested together.[17]

A release gate should run every intended semantic test in fresh kernels, reject empty test reports and runner failures, record the complete file set and hashes, and export failed test details. It should distinguish the declared minimum version from the tested matrix. Windows 15.0.1 evidence in the current summary is useful, but cannot by itself establish all-platform behavior across every supported version. A smaller fast PR suite can coexist with a mandatory comprehensive release run.

8.5 A08: A common head hides several different request contracts

Classification: medium severity; UX and compatibility issue, largely documented.

The guide documents the differences clearly, but their number remains a cost to the user. An integer can mean an exclusive ordinary exponent, a relative amplitude cutoff inside a carrier, reciprocal-log order, inverse-core order, inclusive perturbation depth, or inclusive exponential-sector degree. A term goal can be global for one family and per carrier for another. The output still has the same head.[3]

The right remedy is not to pretend these notions are mathematically identical. Add structured requests, for example:

```
<|"Kind" -> "ExponentCutoff", "Coordinate" -> w,
  "Bound" -> 5, "Boundary" -> "Exclusive"|>
<|"Kind" -> "SectorDepth", "Depth" -> 3,
  "AmplitudeRequest" -> <|...|>|>
```

These are proposed data structures, not current package syntax. Existing shorthand can remain as a convenience, with the expanded request displayed in the result's summary.

Other concrete usability concerns follow from the inspected interface. `Assumptions -> True` is the documented default, whereas many native Wolfram functions use ambient `$Assumptions`; users should not have to infer whether `Assuming` will have the intended effect. The ordinary coordinate routine accepts named directions rather than every native direction syntax. Non-unit `"Power"` refers to an offset-relative observable at finite source points, while the unit power includes the source translation. All three choices should be made explicit in examples and normalized through one entry layer.[3, 4][22]

Numeric application `s[value]` substitutes into the finite expression; it does not evaluate the unknown remainder, prove the supplied value lies in the valid asymptotic region, or return a certified enclosure. `Normal[s]` likewise intentionally discards metadata. These behaviors are documented or visible in the source and are not being called false numerical claims. A safer numerical API would return the approximation together with domain-check status and an explicit “not certified” status unless a separate certificate has been obtained.[4, 3]

Finally, the current README states that version 1.8 renamed the old `PowerLogSeries` head without retaining an alias. That is an intentional compatibility change. Saved expressions and downstream patterns can still break, so a migration function, schema version, and release-note policy are advisable. If semantic-version compatibility is promised, the versioning policy must account for such changes; this audit does not assume an unstated SemVer commitment.[1]

9 Additional proof obligations and unconfirmed probes

9.1 R01: real finite phases do not imply real uncertain phases

In `specialNativeTree`, a sine or cosine node can take a short path when the finite argument approximation is provably real. That path returns the argument’s absolute error without requiring it to tend to zero. For a truly real perturbed argument, the real Lipschitz bound for sine and cosine justifies this. For a possibly complex perturbation, reality of the finite approximation alone is insufficient.[11, `specialNativeTree`]

The formal witness is $z(u) = i/u = 0 + \mathcal{O}(1/u)$. Although the displayed approximation zero is real,

$$|\sin z(u) - \sin 0| = \sinh(1/u) \notin \mathcal{O}(1/u). \quad (15)$$

The broad adapter proves the *source expression* real before final projection. That does not automatically prove every intermediate argument real, and expression-tree arithmetic should not assume it without a contract.

Status: no public source expression was demonstrated here to pass the entire native pipeline and return this bad estimate. Native `Series` may rewrite such expressions into forms taking another guarded route, and final acceptance checks may reject them. Therefore this is a helper-level proof obligation, not a confirmed public incorrect output. A suitable repair would require either reality of the represented argument or a vanishing complex error, with the appropriate complex derivative majorant. That change needs path-specific tests before being shipped.

9.2 R02: real scalar validation differs between entry paths

`seriesRegularOperand` verifies real coefficient polynomials for a scalar added through ordinary arithmetic. By contrast, the variable-independent branch of `seriesIndependentJet` directly constructs a constant jet, and the general observable recursion can reach it for a parameter such as a . The probe compares `SeriesAdd[s,a]` and `SeriesObservable[s,a+z,z]` without real assumptions on a . [6, 5]

The source paths are visibly different. Whether every resulting discrepancy is a violation depends on the intended policy for unproved symbolic constants in forward and observable calculations. The guide describes a real setting, so the policy should be stated and tested, but the supplied bundle does not silently impose a new one. A centralized coefficient-domain predicate or an explicit condition-generating mode would remove the ambiguity.

9.3 R03: unknown native tails need an honest logarithmic degree

The broad native importer acknowledges that a native `SeriesData` tail does not store a separate logarithmic degree. It therefore takes a half-lattice-step loss in the provisional error bound and later demands a sufficiently dominant explicitly computed omitted frontier. This is a careful design.[11]

The older core fallback `fwdSeries`, however, initializes the unknown logarithmic degree to zero and probes one additional native order. If no nonzero omitted coefficient is obtained, the routine can use a new native frontier while leaving degree zero. One must prove that every admitted fallback expression in that situation has the claimed strict analytic Big-O class, or use the same conservative loss policy.[4, lines 452–489]

This is not a demonstrated public counterexample: many elementary power–log expressions use other core routes, and many special functions use the newer adapter. The review target is the remaining fallback surface. A synthetic native-series provider is useful for testing the importer contract, but it would not by itself prove that an ordinary supported built-in expression triggers a current failure.

9.4 Why unconfirmed findings are kept separate

An audit is more useful when it identifies a precise missing proof than when it invents an execution transcript. R01–R03 should become targeted tests and explicit invariants. They should not be turned into release-blocking claims of incorrect public output until the remaining path and scope questions are resolved. They are deliberately not included in the proposed three-edit hardening patch.

10 Mathematical safeguards worth preserving

10.1 Differentiating a Big-O term requires a contract

Let

$$R(x) = x^2 \sin(x^{-4}), \quad x \rightarrow 0^+. \quad (16)$$

Then $R(x) = \mathcal{O}(x^2)$, but

$$R'(x) = 2x \sin(x^{-4}) - 4x^{-3} \cos(x^{-4}). \quad (17)$$

At $x_n = (2\pi n)^{-1/4}$, the first term vanishes and $R'(x_n) = -4x_n^{-3}$. Thus R' is not $\mathcal{O}(x)$. A generic magnitude-only remainder cannot be differentiated by subtracting one from its exponent.

The package’s derivative-order declaration and refusal without sufficient evidence address exactly this issue. Automatically granting derivative control to a Poincaré expansion merely because native `Series` generated it would undo a major safety property. Exact finite expressions, explicit conormal derivative contracts, and native formal-series calculus must remain distinct notions.[5, 11, 3]

10.2 Exponentiating a relative approximation can be wrong

If $g = A + R$ with $R \rightarrow 0$, then $e^g = e^A(1 + \mathcal{O}(R))$, with an eventual constant justified by the local exponential derivative. But $g \sim A$ does not suffice. For $A = x$ and $R = \log x$, one has $g/A \rightarrow 1$ while

$$\frac{e^g}{e^A} = x \rightarrow \infty. \quad (18)$$

The relevant requirement is *small absolute phase error*, not small relative error in the phase. The explicit series exponential and source-coordinate reconstruction guard this distinction. Keeping an exact inverse core can avoid an otherwise invalid exponential amplification of a finite inverse approximation.[5, 12]

10.3 Oscillatory coefficients are not invertible units by default

An expression such as $u^\alpha \sin(\log u)$ has a bounded oscillatory coefficient but infinitely many zeros approaching the endpoint. Treating the sine factor as an eventually nonzero leading constant would invalidate division and relative-error estimates. Separate oscillatory coefficient handling and absolute envelopes are appropriate; a claimed inverse or reciprocal needs stronger sign or separation evidence. The inspected native-sector logic explicitly avoids promoting bounded oscillations to nonzero constants.[11]

10.4 A zero truncated residual is not automatically exact termination

Formal cancellation below a cutoff proves consistency to that cutoff, not equality to the original function. The inverse construction distinguishes an exact model from a finite forward jet and attempts an explicit exact-termination certificate only on eligible reliable blocks. The incremental Newton route independently checks the residual at its claimed precision. These are safeguards to preserve during performance optimization.[4, 8]

10.5 Exact interval certification: a separate, narrower achievement

The certificate subsystem uses rational intervals with directed dyadic rounding and explicit series tails for elementary exponentials and logarithms. Approximate arithmetic is used for seeds, but acceptance depends on exact enclosures. The code checks the entire verification interval against retained source-domain conditions and separates the derivative from zero. It states that it certifies the stored explicit function on that interval, not an unspecified input-remainder family or a global inverse branch.[9]

Proposition 10.1 (Residual containment certificate). *Let $F = f - y$ be continuously differentiable on an interval I . Suppose $c \in I$, $|F(c)| \leq \varepsilon$, and f' has constant sign with $|f'(x)| \geq \mu > 0$ on I . If $[c - \varepsilon/\mu, c + \varepsilon/\mu] \subseteq I$, then F has exactly one root in I , and its distance from c is at most ε/μ .*

Proof. Assume $f' > 0$; the decreasing case is analogous. With $r = \varepsilon/\mu$, the mean-value theorem gives $F(c - r) \leq F(c) - \mu r \leq 0$ and $F(c + r) \geq F(c) + \mu r \geq 0$. Continuity gives a root between these points. Strict monotonicity on I gives uniqueness. The enclosure immediately gives the distance bound. $\square$

The implementation also permits exact endpoint signs when the residual bracket is not contained, and then sharpens the enclosure using the signed derivative interval. For example, for $f(x) = x + x^2$, $y = 1$, $I = [1/2, 3/4]$, and $c = 5/8$,

$$F(c) = 1/64, \quad f'(I) = [2, 5/2], \quad r = 1/128. \quad (19)$$

The signed mean-value correction gives the independent rational enclosure

$$x_* \in \left[\frac{5}{8} - \frac{1}{128}, \frac{5}{8} - \frac{1}{160} \right] = \left[\frac{79}{128}, \frac{99}{160} \right]. \quad (20)$$

This is a mathematical illustration of the certificate argument, not a literal recorded output from `InverseCertificate`.

For logarithms, the underlying positive-argument reduction uses a convergent atanh-type series with a geometric tail; exponentials are reduced to a small positive argument and bounded by explicit series tails before reconstruction. Directed rounding must enclose, not merely approximate, every intermediate arithmetic operation. This subsystem is sufficiently valuable to justify independent unit tests of its primitives and eventual independent verification of its certificates.[9]

11 Performance analysis and a fair benchmark program

11.1 Separate representation complexity from coefficient complexity

Several different size measures coexist: the number of retained weights, the number of multi-indices contributing to those weights, the degree and coefficient size of each logarithmic polynomial, the Fourier-mode count, the number of exponential sectors, and the size of the provenance graph. A cutoff or term goal constrains only part of this state. A02 is an extreme example in which a representation conversion, not coefficient generation, dominates cost.

For positive fixed gaps $\delta_1, \dots, \delta_m$, the number of nonnegative multi-indices with $\mathbf{k} \cdot \boldsymbol{\delta} < H$ grows on the scale of the simplex volume

$$\frac{H^m}{m! \delta_1 \cdots \delta_m} \quad (21)$$

as H becomes large, up to boundary effects. This geometric estimate explains why many generators or very small gaps can be expensive even when equal weights collapse to far fewer output blocks. It is a complexity model, not a timing prediction for the package.

Grouped Lagrange evaluation can exploit collisions; Newton can avoid explicitly enumerating every multi-index in suitable regimes. Their relative value depends on sparsity and coefficient growth. An automatic selector should use observed state statistics and bounded pilot work, not a universal rule that one method is always faster. The existing multiple implementations are also useful differential test oracles.^[8]

11.2 What the repository’s recorded benchmarks actually support

Recorded workload	Before (s)	After (s)	Approx. ratio
100 translated coordinate inversions	0.12115	0.00418	29.0×
Fourier coefficient merge	0.19112	0.09396	2.0×
200 × 200 Fourier product, weight cutoff 4	1.28634	0.00776	165.8×
Public Fourier inverse case	0.02514	0.02325	1.08×

These are *repository-reported* medians, not new audit measurements. The cited record describes one warm-up and three measured runs in fresh native-kernel benchmarking conditions. The dramatic low-level product improvement did not translate to a similarly dramatic public inverse improvement in the cited example. Earlier records also distinguish noise-sensitive end-to-end measurements from clear local gains. That restraint should be retained in future performance claims.^[14]

11.3 Recommended workload matrix

A useful benchmark suite covers ordinary Taylor inversion as a native baseline; sparse irrational gaps; dense rational lattices; many resonant generators; high logarithmic degree; repeated refinement; translated and signed endpoints; Gamma/Barnes exact-core observables; mixed carriers; and Fourier frequency growth. Each workload needs both a coefficient or residual oracle and an error-contract oracle.

Report fresh-process load time separately from cold computation, and cold computation separately from warm repeated runs. Record kernel version, operating system, source commit, loader mode,

options, output shape, failure/timeout status, retained blocks, generated candidates, and peak-memory measurement methodology. A before/after comparison must run the same mathematical request under both implementations. Do not silently loosen a cutoff, drop a logarithmic factor, or strip provenance in only one branch to obtain a favorable ratio.

The bundled harness is deliberately small and diagnostic. It includes a matched sine-inverse case, a generalized inverse comparison with `AsymptoticSolve`, an `Erfc` comparison, and the moderate rational-lattice case. It is not a publication-ready benchmark result. Later cases in the same process are explicitly not labelled fresh-process cold timings.

12 API and representation redesign proposal

12.1 A shared result contract

Keep the common `GeneralizedSeries` surface, but give every result a schema describing its mathematical obligations. At minimum, separate the finite expression, coordinate approach, coefficient algebra, support/truncation request, error class, derivative evidence, branch evidence, and provenance identifier. Fields that do not apply should be explicitly unavailable rather than overloading a similarly named field with a different meaning.

A proposed representation could expose

```
<|"SchemaVersion" -> 2,
  "Approach" -> <|"Variable" -> y, "Point" -> Infinity,
    "Side" -> "FromBelow", "Coordinate" -> 1/y|>,
  "CoefficientAlgebra" -> "PolynomialLog",
  "Truncation" -> <|"Kind" -> "ExponentCutoff",
    "Bound" -> 5, "Boundary" -> "Exclusive"|>,
  "Error" -> <|"Kind" -> "AsymptoticBigO",
    "Scale" -> ..., "DerivativeOrder" -> 0|>,
  "Evidence" -> <|"RealBranch" -> ..., "Exactness" -> ...|>,
  "ProvenanceID" -> ...|>
```

This is a design sketch, not an implemented API. The error record should distinguish an asymptotic class from an explicit pointwise majorant and from an interval certificate at a particular target. A field named `"Certified"` should never be inferred merely from a symbolic residual or an exact finite model.

12.2 Centralize invariant enforcement

Every public operation, including nested observables, should reach the same low-level checks for real coefficients, positive bases of noninteger powers, nonzero reciprocal bases, compatible approaches, and allowable remainder transport. This is the architectural lesson of A01. A direct operation and a semantically equivalent nested form should differ only in diagnostic context, not in whether an invariant is enforced.

A three-valued proof outcome is preferable to a Boolean overloaded with failure: proved, disproved, and unresolved. Attach a reason to unresolved results, such as timeout, unsupported syntax, or insufficient precision. For an inverse branch, candidate generation, local validation, and completeness should remain separate outcomes, as the current branch module already does.^[10]

12.3 Improve diagnostics without weakening conservatism

Expose the actual selected constructor and transformations, not only the originally requested method. A diagnostic trace should explain the normalized coordinate, detected leading scale, attempted algorithms, precision requested from each operand, and the particular missing proof. Keep such traces bounded and optional; otherwise they become another provenance-size problem.

For failures caused by insufficient input order, give the maximum justified output cutoff and the extra source precision needed. For an unsupported coefficient ring, name the ring encountered and the supported alternatives. For native special-function failures, distinguish native expansion failure, real-domain failure, unsupported expression-tree transport, and inability to dominate the native tail by the proposed omitted frontier. The inspected code already has many differentiated failure tags; the next improvement is a coherent public taxonomy rather than collapsing them into a generic message.[4, 11, 6]

12.4 Numerical evaluation should expose what it discards

A proposed `EvaluateApproximation` wrapper could return a finite numeric value, the evaluated asymptotic scale, domain-check status, and certificate status. An evaluated scale alone is *not an error bar*, because a Big-O statement lacks a supplied constant and validity threshold. Where a pointwise bound or `InverseCertificate` is available, include its explicit interval and function scope separately.

Do not force approximate numeric inputs into the exact coefficient algebra by silently rationalizing them. The package's exact-input restriction is an intentional design choice. A convenience layer could accept a numerical target for evaluation or an explicitly declared parameter approximation, but should keep those modes distinct from exact symbolic construction.[3, 17]

13 Missing functionality and development opportunities

13.1 A safe generalized-series integration operation

The current public function overview includes differentiation but no general series-integration operation. This is an opportunity to extend the algebra, not evidence that the native `AsymptoticIntegrate` facility is missing.[3, 25]

For a logarithmic monomial and $\beta \neq -1$, a natural coefficient rule is

$$\int u^\beta (\log u)^k du = u^{\beta+1} \sum_{j=0}^k \frac{(-1)^j k!}{(k-j)! (\beta+1)^{j+1}} (\log u)^{k-j} + C. \quad (22)$$

At resonance $\beta = -1$ the result is $(\log u)^{k+1}/(k+1) + C$. The remainder rule requires a separate theorem about the chosen definite or indefinite integral and its normalization. For instance, integrating a bound from 0 is straightforward only when the error is integrable there. Transformed coordinates also require the correct Jacobian. A serious implementation must expose the constant of integration and the endpoint convention, not merely integrate displayed terms and carry an unproved Big-O suffix.

13.2 Uniform parameter regimes and transition points

The current real-domain and parameter checks do not constitute a general uniform asymptotic theory. When a leading coefficient tends to zero, two exponents cross, a gap tends to zero, or saddle/branch

structures change, a fixed-parameter expansion can become nonuniform. A future API should state whether assumptions define only a pointwise parameter regime or a region with uniform constants.

An initial practical extension would partition parameter space into regions with a stable leading scale and report transition boundaries explicitly. More ambitious uniform approximations should be separate constructors with their own error theorems. It would be unsafe to advertise all parameter values merely because a native special-function head is recognized. The guide's existing warnings about supported regimes are the correct starting point.[3, 11]

13.3 Multivariate and coupled implicit systems

The current inverse API selects a single source and target variable. Wolfram's `AsymptoticSolve` already supplies a native baseline for systems. A package extension could add multivariate supports and a valuation cone, but then order is generally partial, Newton polygons/polyhedra enter, and branch/uniqueness evidence becomes more complicated. This should be an explicit new algebra rather than an unnoticed generalization of one-dimensional sort order.[3, 23]

13.4 Broader coefficient algebras and transseries

The existing logarithmic, exact-core, flat-sector, and Fourier families provide a useful foundation. They should not be described as a universal closure under arbitrary nested exponentials, iterated logarithms, complex sectors, Stokes transitions, or resurgent operations. The current guide gives family-specific boundaries, and several operations explicitly reject unsupported scales.[3, 5]

A modular coefficient-algebra interface can support future rational functions of logarithms, nested slow variables, and additional oscillatory structures. Each extension needs an order relation, zero/sign logic, differentiation rules, composition conditions, truncation semantics, and an error-envelope interpretation. Adding a parser pattern for a new expression is not the same as implementing that algebra.

13.5 Extending numerical certification

Useful next steps include interval subdivision when dependency inflation prevents derivative separation, tighter polynomial enclosures, explicit bounds for additional elementary/special functions, and certificate verification independent of the approximation generator. A proof-checking layer could validate rational arithmetic, tail inequalities, domain facts, and the residual-to-root theorem with a much smaller trusted surface than the full symbolic engine.

The distinction between certifying the stored explicit model and certifying a family with unspecified input error must remain visible. Extending the latter requires a pointwise interval envelope for that family and its derivative, not only an asymptotic exponent pair. It also requires a clear statement of which branch, interval, and target are covered.[9, 3]

13.6 Distribution and compatibility

Keep modular sources canonical and regenerate the standalone file. The guide already supplies pinned and offline loading instructions and documents the observed direct-HTTP `Get` issue; proposing those facilities as wholly new would be inaccurate. The remaining opportunity is release packaging with explicit versioned documentation, checksums, migration guidance, and a declared semantic test matrix. Prefer a pinned source or release in reproducible examples rather than a floating `main` URL.[3, 15, 1]

14 Test strategy and prioritized roadmap

14.1 Test mathematical invariants, not only example strings

Every supported construction should be tested for finite coefficients, retained order, coordinate and domain, derivative eligibility, and exactness classification. Differential tests between Lagrange, grouped Lagrange, and Newton are valuable, but they are not independent when all routes share the same low-level guard or error converter. Include direct coefficient identities, exact inverse examples, original-function residuals, and independently derived error bounds.[8, 17]

Metamorphic tests should compare a direct power with the same power inside a sum or composition; compare refinement in one step with refinement in several steps; check truncation idempotence and monotonicity; test translation and signed-coordinate equivariance; and ensure budget failures never silently manufacture missing coefficients. Mutation tests that remove a branch guard, a log-degree maximum, or an input-order cap are useful for checking whether the suite would detect the intended class of defect.

Native importing needs adversarial expression-tree tests: unknown logarithmic tails, exact offsets plus uncertain carriers, cancellation of carriers, complex intermediate phases, parameter sign changes, and oscillatory zeros. Tests for a mock provider should be labelled adapter-contract tests, not evidence of behavior of an actual native special function.

14.2 Roadmap with acceptance criteria

Priority	Work	Acceptance criterion
P0	A01 shared real-power guard; A02 dense-export cap in both converters	Direct and nested branch tests agree; sparse high-denominator construction returns normally with unavailable native export; no loss of supported exact-zero behavior.
P0	Native validation of the proposed patch	Full semantic suite finishes in fresh kernels with nonempty reports; modular and rebuilt standalone outputs agree; source identities and failures are archived.
P1	A03 early termination; targeted frontier optimization	Exact coefficient and residual equality retained; operation counts or native timings demonstrate reduced work without changing requested precision.
P1	Budget taxonomy and output preflight	Every exhausted budget is identified; small input cannot trigger an unbounded optional conversion; partial results retain sound errors.
P1	Schema, validated import, compact provenance graph	Malformed payloads are rejected on external import; repeated shared computation has bounded graph growth and scalable serialization.
P1	Resolve R01–R03	Either an explicit invariant proof closes each path or a minimal native regression and repair are recorded.

Priority	Work	Acceptance criterion
P2	Structured truncation requests, capability reporting, compatibility/migration	Every family reports its coefficient algebra, coordinate, cutoff meaning, and supported operations in one stable interface.
P2	Generalized integration and stronger interval certificates	Coefficient identities and endpoint/error theorems accompany implementation; numerical certificate scope remains explicit.
P3	Uniform parameter regions, multivariate supports, broader transseries	Each extension has a separately specified algebra and validation corpus; no universal-support claim is inferred from syntax recognition.

Priorities denote dependency and impact, not promised delivery dates or estimated implementation times.

15 Conclusions

The repository’s strongest engineering choices are worth retaining: exact real-weight arithmetic, complete collision layers, multiple inverse algorithms, explicit input-precision caps, distinct symbolic/numerical/certified evidence, derivative-contract checks, exact phase cores, and honest composite envelopes. Its extensive documentation and cautious validation record make a source audit substantially more productive than an undocumented package would be.[3, 8, 5, 11, 9, 14]

The most urgent defects are not an absence of advanced special functions. They are failures of common infrastructure: a real-power invariant that is enforced at one entry point but bypassed at another, and a sparse object that eagerly builds an enormous dense native representation. A small recurrence optimization is also mathematically clear. These are concrete targets for the supplied patch and regression bundle.

The durable product direction is a dependable generalized asymptotic object model layered over Wolfram Language’s existing strengths. A validated common contract, resource-aware representation adapters, shared provenance, and reproducible release evidence will make every current and future family more useful than adding breadth without those foundations.

A Review coverage and source map

All repository references below point to the pinned commit. Line ranges refer to the original modular files, not to the generated standalone file. Some long connector responses were truncated; the audit uses the visible central paths and does not claim unseen ranges were reviewed.

Source	Review focus
<code>AsymptoticInverse/Kernel/AsymptoticInverse.wl</code>	Public/core definitions; jet arithmetic; composition recurrence (251–263); powers (369–387); native fallback (452–489); coordinate construction; forward native conversion (647–660); inverse construction/-precision caps; inverse native conversion and object accessors.
<code>Kernel/SeriesOperations.wl</code>	Representation extraction and construction; binary/unary operations; direct power guard; generic observable recursion; composition; truncation; derivative contract.
<code>Kernel/SeriesArithmetic.wl</code>	Held normalizer, operator upvalues, scalar validation, guarded fallback, and nested dispatch.
<code>Kernel/SeriesEnvelopeArithmetic.wl</code>	Absolute composite errors, approach compatibility, resource checks, result/provenance construction.
<code>Kernel/IncrementalInverse.wl</code>	Equal-weight frontier layers, polynomial caches, grouped Lagrange and Newton precision checks.
<code>Kernel/InverseCertificates.wl</code>	Exact interval primitives; exponential/logarithmic tails; domain checks; residual/derivative certificate and stated scope.
<code>Kernel/InverseFunctionBranches.wl</code>	Candidate generation, unresolved proof outcomes, local validation, monotonicity and completeness distinctions.
<code>Kernel/NativeSpecialFunctions.wl</code>	Native candidate dispatch, expression-tree error transport, real projection, sector extraction, truncation and acceptance.
<code>Kernel/SourceCoordinates.wl</code>	Exact source charts, carrier reconstruction, exponential error guard, provenance and source-power conventions.
<code>Kernel/SpecialFunctionAdapters.wl</code>	Erfc/Gamma model bounds and provenance; local threshold adapters; explicit distinction between exact and finite asymptotic models.
<code>Documentation/UserGuide.md</code> ; root <code>README.md</code>	Current public API, family-specific cutoff meanings, requirements, loading and migration. Broader family coverage in the report is guide-based rather than a line-by-line proof audit of every family implementation.
<code>Tests/SeriesArithmetic.wlt</code>	Representative coefficient, branch, precision, compatibility and replay tests; not a full review or execution of the test tree.
<code>validation/README.md</code> ; <code>docs/development/README.md</code> ; <code>.github/workflows/standalone.yml</code>	Historical validation scope, benchmark interpretation, canonical/generated source process, inspected CI behavior.

Paths abbreviated as `Kernel/...`, `Tests/...`, or `Documentation/...` are relative to `AsymptoticInverse/`. Other family modules were inventoried and contextualized through the guide, but are not

included in this table as fully reviewed implementations.

B Independent inverse-coefficient oracle

For the real germ $f(x) = x + x^{1+d}$ with $d > 0$, write its inverse as $x = yW(t)$ with $t = y^d$. Then

$$W + tW^{1+d} = 1. \quad (23)$$

The first coefficients are

$$\begin{aligned} W(t) = & 1 - t + (d+1)t^2 - \frac{(d+1)(3d+2)}{2}t^3 \\ & + \frac{(d+1)(2d+1)(4d+3)}{3}t^4 \\ & - \frac{(d+1)(5d+2)(5d+3)(5d+4)}{24}t^5 + \mathcal{O}(t^6). \end{aligned} \quad (24)$$

They also follow from the coefficient formula

$$c_n = \frac{(-1)^n}{n!} \prod_{j=1}^{n-1} (1 + nd + j), \quad n \geq 1. \quad (25)$$

The included checker substitutes these coefficients into $W + tW^{1+d} - 1$, expands the power independently using the binomial identity, and checks every coefficient through degree five is exactly zero as a polynomial in d . This is a small but genuinely independent algebraic oracle for generalized real exponents. It does not use a native Wolfram kernel or call the package.

C Bundle contents and execution status

Artifact	Status / purpose
<code>article.tex</code> , <code>article.pdf</code>	This report and its compiled PDF.
<code>code/patch_core.py</code>	Source-identity-guarded three-edit patch generator. Python editing logic tested on anchor fixtures; not applied to a full checkout here.
<code>code/independent_checks.py</code>	Executed exact mathematics and allocation planner; not a Wolfram emulator.
<code>code/run_native.wl</code>	Fresh-kernel audit regression runner. Not executed here.
<code>tests/AuditRegressions.wlt</code>	Nine native regression/control specifications, including two nested A01 cases expected to fail before the patch. Not executed here.
<code>tests/PatchedOnly.wlt</code>	Two guarded high-denominator tests; do not enable on the unpatched source. Not executed here.

Artifact	Status / purpose
<code>code/inspection_probes.wl</code>	Bounded exploratory probes for scalar consistency, raw object invariants, and provenance growth. Not executed here.
<code>code/benchmark_native.wl</code>	Version/hash-aware, time- and memory-bounded diagnostic comparison harness. Not executed here.
<code>tests/test_python_artifacts.py</code>	Eight executed tests of planner and fail-closed patch tooling.
<code>evidence/independent-checks.json</code> and logs	Executed mathematical results and Python test output. No fabricated native results.
<code>evidence/source-manifest.json</code> , <code>evidence/findings.json</code>	Pinned source identities, scope, and machine-readable finding status.
<code>README.md</code> , <code>SHA256SUMS</code>	Build/run instructions, limitations, and bundle file checksums.

C.1 Applying the proposal in a disposable checkout

The patch generator writes a new output directory outside the checkout; it never edits the repository in place. First obtain the pinned commit, then run

```
python code/patch_core.py --repository /path/to/Asymptotic \
--output /path/to/new-audit-patch-directory
```

Review the emitted diff, apply it to a disposable branch, regenerate the standalone file with the repository's builder, and run both the audit subset and the complete upstream semantic suite in native kernels. A02 uses a fixed 20,000-slot safety cap; threading a configurable budget and making conversion lazy are separate follow-up work.

C.2 Native runner safeguards

Set `ASYMPTOTIC_AUDIT_PACKAGE` to the absolute local source path. The patched-only suite is enabled only when `ASYMPTOTIC_AUDIT_PATCHED=1` and the loaded source text contains the A02 patch marker. The tests add a memory guard. These checks reduce the chance of accidentally executing the deliberately expensive unpatched example; they do not substitute for reading the source and test instructions.

The runner rejects empty or incomplete test reports and writes the resulting version, platform, entry-file hash, counts, and report. When loading the modular entry, an entry-file hash alone does not identify all companions: record the pinned commit and full module manifest as part of release acceptance. The README supplies separate POSIX and PowerShell examples.

C.3 Building the article

Run `latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex`. The document is self-contained and requires standard LaTeX packages; no external image or font files are distributed. The final PDF was rendered for layout inspection. None of the report's source-code findings relies on OCR or a screenshot of code.

References

- [1] Asymptotic contributors. *Repository README*, version 1.8.0 snapshot. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/README.md>.
- [2] Asymptotic contributors. *Pinned repository tree*. <https://github.com/VladimirReshetnikov/Asymptotic/tree/07a9781212beb2eeb9ff16aa625b50ac27974078>.
- [3] Asymptotic contributors. *AsymptoticInverse User Guide*, especially usage, directions, input remainders, and the family-specific cutoff table. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Documentation/UserGuide.md>.
- [4] Asymptotic contributors. *Canonical kernel: AsymptoticInverse.wl*. Git blob [ea9eaf4a11130e922ac4fb3faae37fd8e8d29643](https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/AsymptoticInverse.wl). <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/AsymptoticInverse.wl>.
- [5] Asymptotic contributors. *SeriesOperations.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesOperations.wl>.
- [6] Asymptotic contributors. *SeriesArithmetic.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesArithmetic.wl>.
- [7] Asymptotic contributors. *SeriesEnvelopeArithmetic.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl>.
- [8] Asymptotic contributors. *IncrementalInverse.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/IncrementalInverse.wl>.
- [9] Asymptotic contributors. *InverseCertificates.wl*, exact interval primitives and certificate scope. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseCertificates.wl>.
- [10] Asymptotic contributors. *InverseFunctionBranches.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseFunctionBranches.wl>.
- [11] Asymptotic contributors. *NativeSpecialFunctions.wl*, structured native-series transport and acceptance. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>.
- [12] Asymptotic contributors. *SourceCoordinates.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SourceCoordinates.wl>.
- [13] Asymptotic contributors. *SpecialFunctionAdapters.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SpecialFunctionAdapters.wl>.
- [14] Asymptotic contributors. *Validation record*. Focused results and benchmark caveats are historical claims of this record, not new audit runs. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/README.md>.
- [15] Asymptotic contributors. *Development notes and standalone build process*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/docs/development/README.md>.

- [16] Asymptotic contributors. *Standalone package freshness workflow*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/.github/workflows/standalone.yml>.
- [17] Asymptotic contributors. *SeriesArithmetic.wlt*, representative native regression tests. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Tests/SeriesArithmetic.wlt>.
- [18] Wolfram Research. *Series*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [19] Wolfram Research. *SeriesData*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [20] Wolfram Research. *InverseSeries*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [21] Wolfram Research. *ComposeSeries*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [22] Wolfram Research. *Asymptotic*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [23] Wolfram Research. *AsymptoticSolve*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [24] Wolfram Research. *AsymptoticLess*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>.
- [25] Wolfram Research. *AsymptoticIntegrate*. Official reference, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>.